Chapter 9: Unsupervised Learning Techniques

Hands-On Machine Learning, 3rd edition (Aurelien Geron)

Giacomo Fiumara

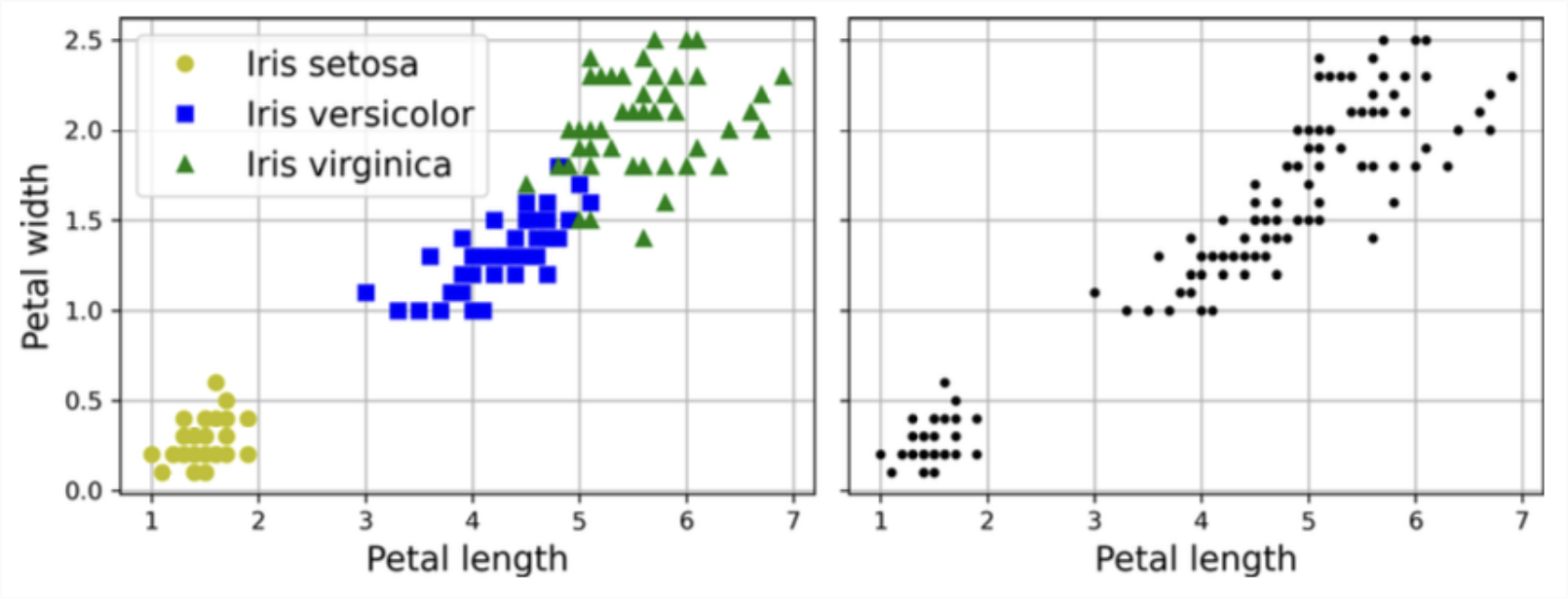
Overview

- Unsupervised learning uses unlabeled data; clustering, density estimation, and anomaly detection.
- Core algorithms covered are k-means, DBSCAN, Gaussian Mixture Models, and Bayesian Gaussian mixtures.
- Applications include image segmentation, semi-supervised learning, feature engineering, and novelty detection.

Why unsupervised learning

- Most available data is unlabeled, so scalable learning often requires methods that exploit structure without targets.
- Clustering groups similar instances, anomaly detection finds unusual cases, and density estimation models data generation.
- These tools support exploration, preprocessing, monitoring, and downstream supervised learning.

Classification vs clustering



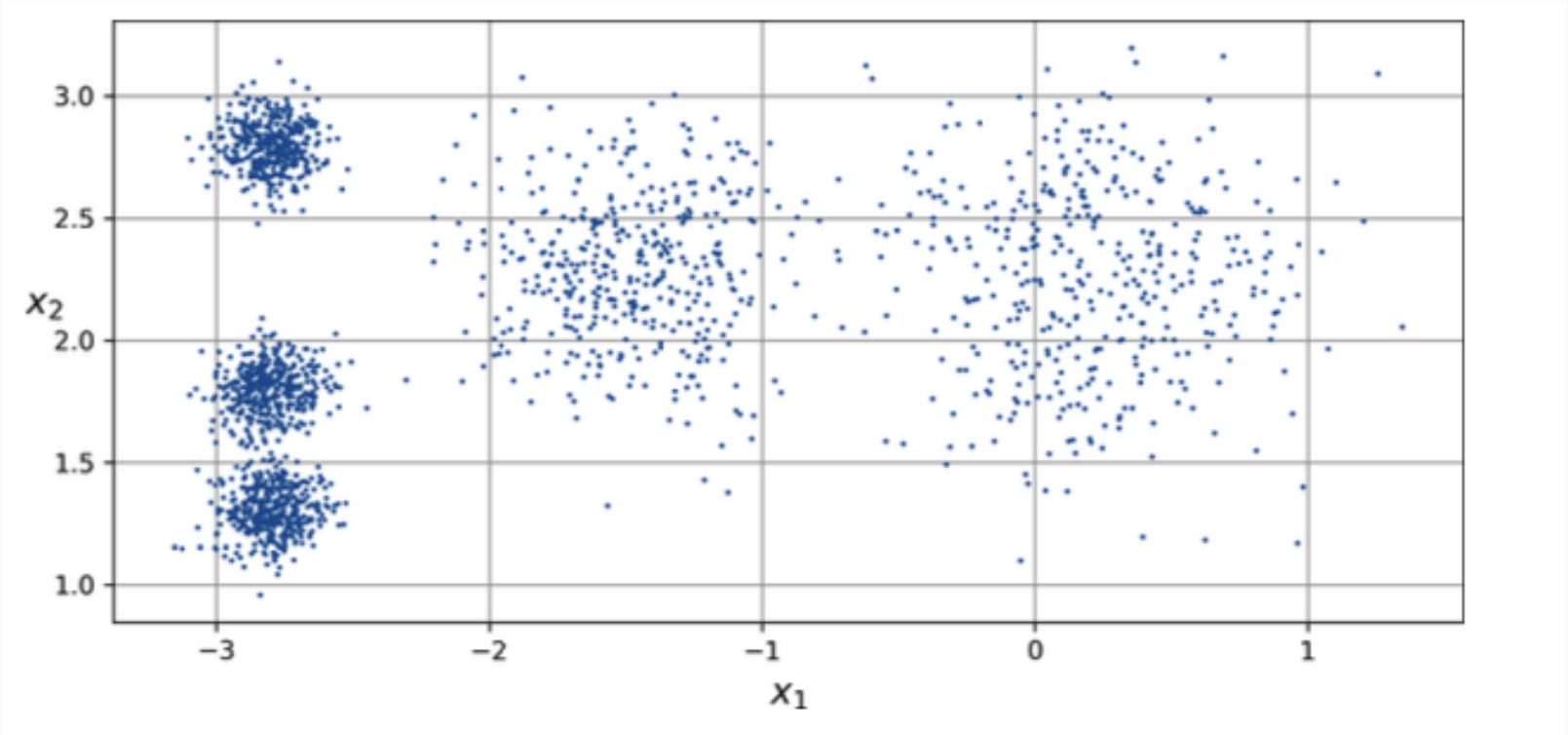
Clustering basics

- Clustering assigns instances to groups of similar items, but unlike classification the groups are not given in advance.
- Different algorithms encode different notions of a cluster: centroid-based, density-based, or hierarchical.
- Choice of algorithm depends on shape, density, scale, and intended downstream use.

Clustering applications

- Customer segmentation helps tailor products, marketing, and recommendations to groups with similar behavior.
- Cluster affinities can reduce dimensionality or serve as engineered features for another model.
- Other uses include search, image segmentation, anomaly detection, and semi-supervised label propagation.

Unlabeled blobs dataset



k-Means

k-means big picture

- k-means looks for k centroids and assigns each instance to its nearest centroid.
- It alternates between label assignment and centroid update until centroids stop moving.
- The algorithm is fast and often converges in only a few iterations on blob-like data.

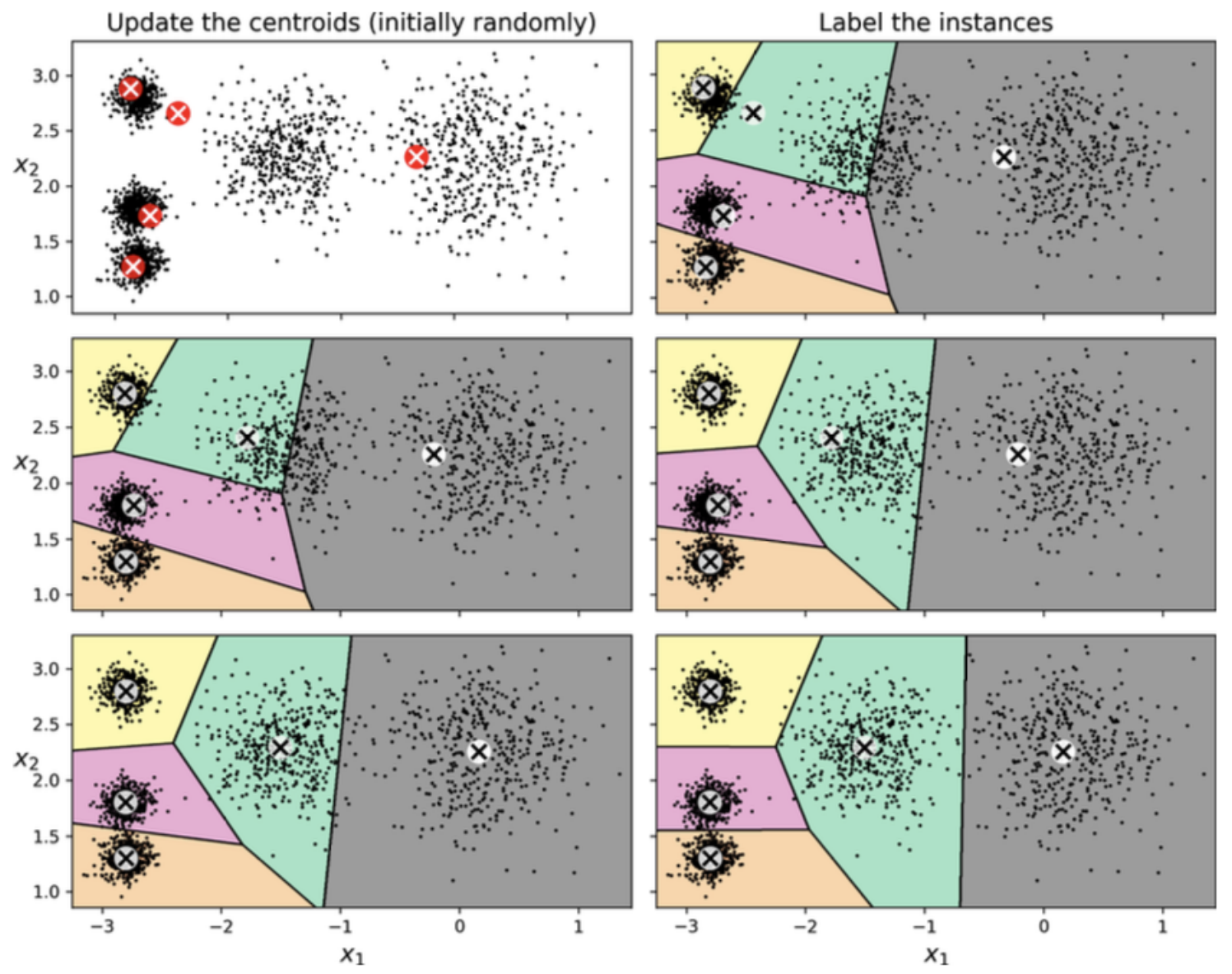
Hard vs soft clustering

- Hard clustering assigns each instance to exactly one cluster; this is the default k-means output.
- Soft clustering uses scores such as distances or affinities to quantify membership strength for every cluster.
- The transform method yields a k-dimensional representation that can support nonlinear dimensionality reduction.

k-means algorithm details

- Start with random or user-provided centroids, assign points to nearest centroid, then recompute centroid means.
- Each iteration decreases the mean squared distance to the nearest centroid, so convergence is guaranteed.
- Practical complexity is roughly linear in number of instances, clusters, and dimensions on structured data.

k-means iterations



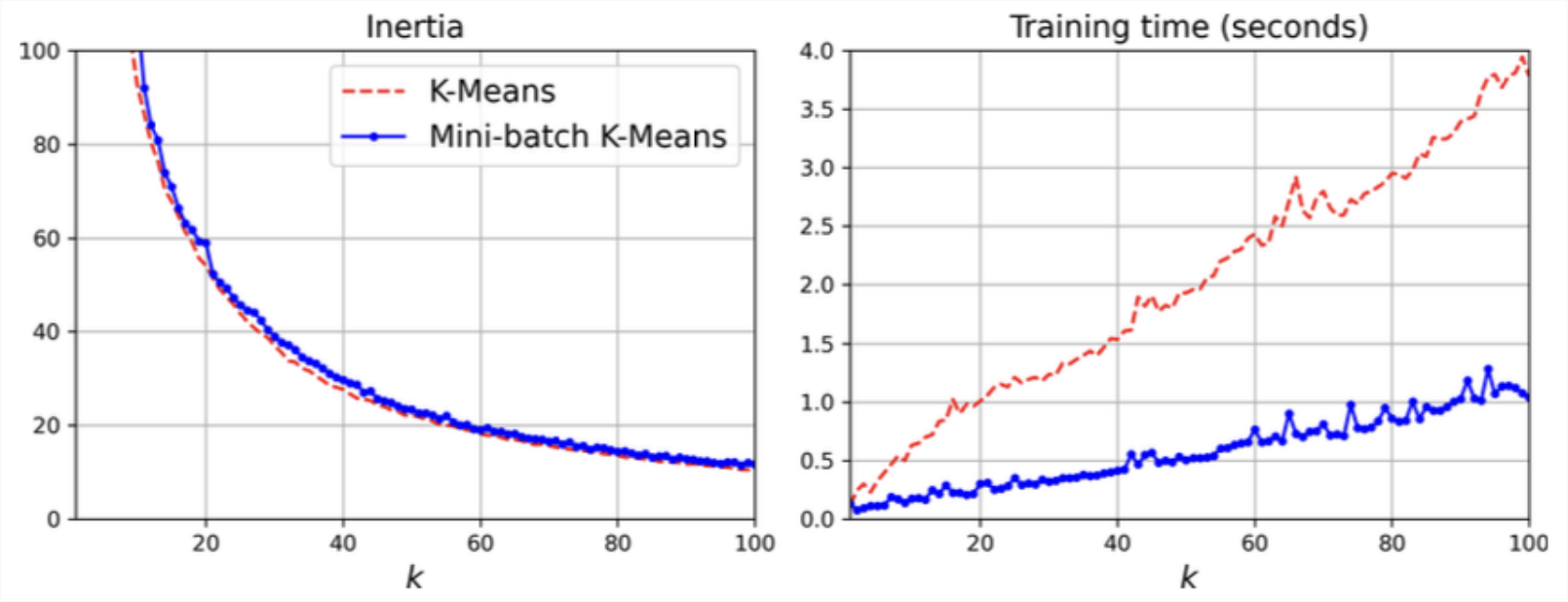
Initialization and inertia

- Random starts may lead to poor local optima, so k-means is usually run multiple times and the best result is kept.
- Inertia is the sum of squared distances from each instance to its closest centroid; lower is better.
- k-means++ improves seeding by choosing distant centroids, greatly reducing bad initializations.

Accelerated variants

- Elkan k-means prunes distance computations using triangle inequality bounds, which can help on some datasets.
- Mini-batch k-means updates centroids using small batches, enabling much faster training on large datasets.
- The speedup often comes with slightly worse inertia than full k-means.

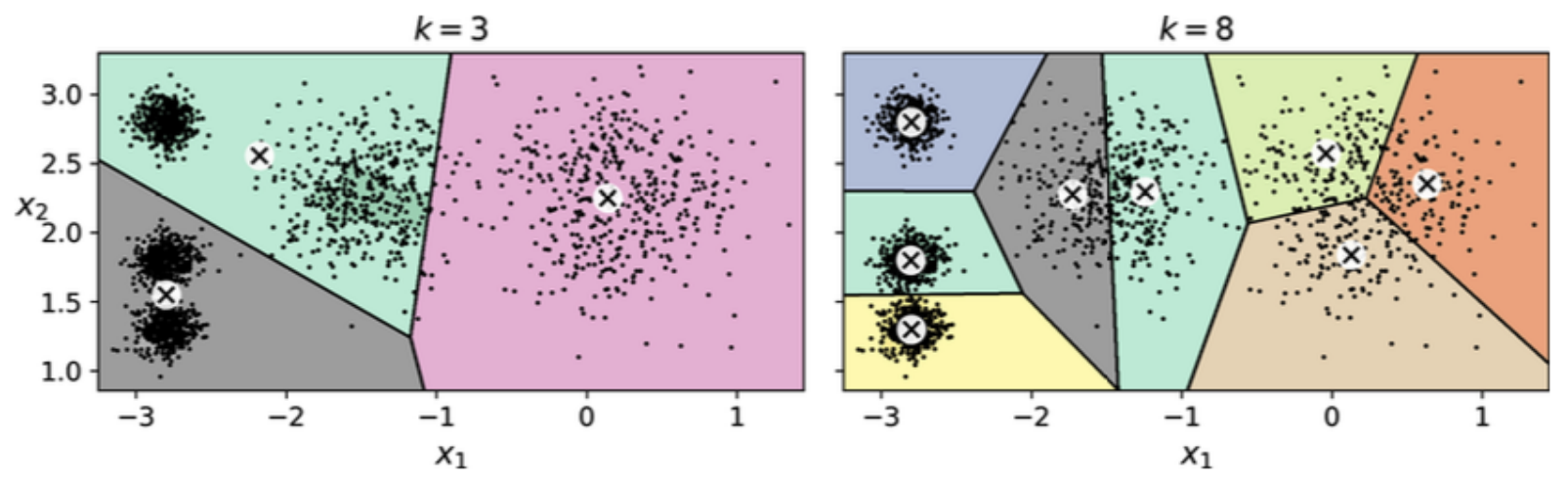
Mini-batch vs k-means



Choosing k

- The number of clusters is a hyperparameter, and bad choices can merge distinct groups or split valid ones.
- Inertia always decreases as k grows, so it cannot be used alone to pick k.
- The elbow method and silhouette analysis provide more informative guidance.

Bad choices for k



Why choosing k matters

- In k-means, the number of clusters k must be fixed before training.
- If k is too small, distinct groups are merged; if k is too large, good clusters are split into smaller pieces.
- Two practical tools are frequently used: the **elbow method** and the **silhouette score**.

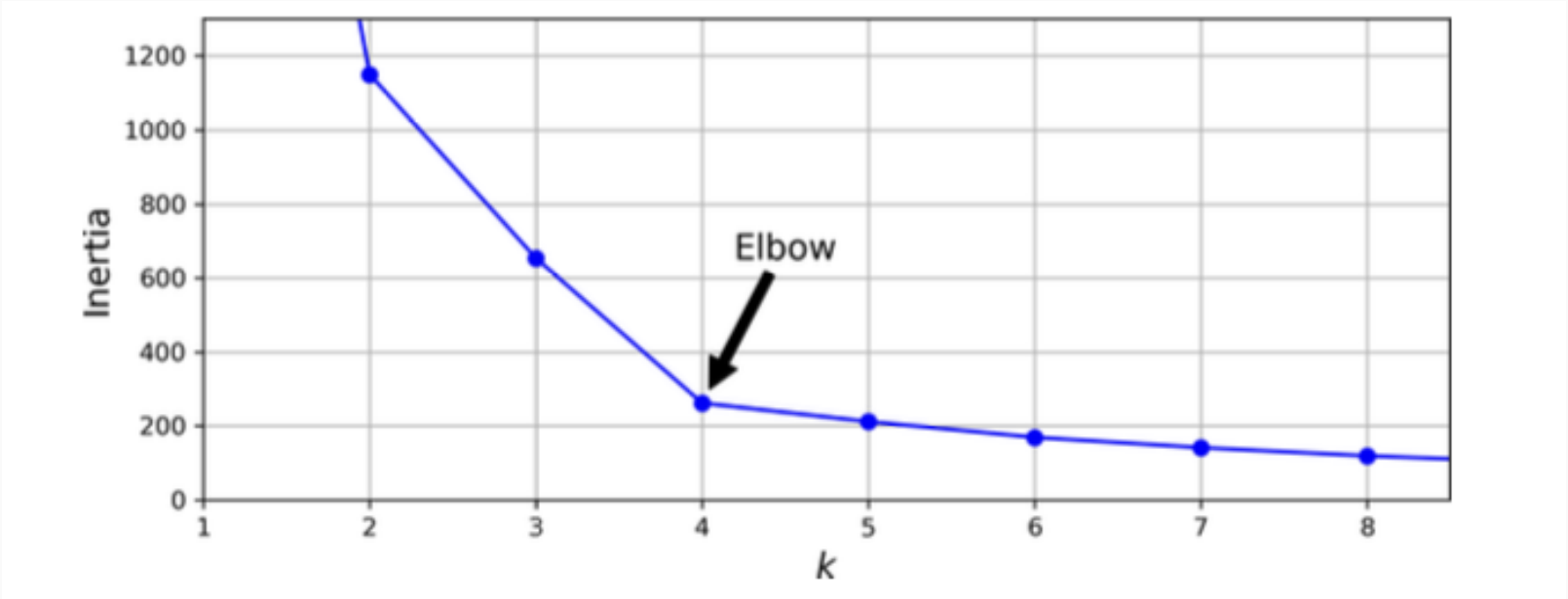
Why inertia alone is not enough

- k-means minimizes the inertia, i.e., the sum of squared distances from each point to its nearest centroid.
- Inertia always decreases when k increases, because more centroids place each point closer to some center.
- Therefore, choosing the model with the lowest inertia would always favor larger k, even when the clustering is not meaningful.

Elbow method: basic idea

- Train k-means for several values of k and plot inertia as a function of k.
- Look for the point where the curve changes from a steep drop to a much flatter decrease.
- This inflexion point is called the elbow: before it, extra clusters help a lot; after it, gains are marginal.

Elbow method



Elbow method: interpretation

- A clear elbow suggests a reasonable compromise between model simplicity and cluster compactness.
- Inertia drops quickly up to $k = 4$ and then decreases more slowly, so the elbow appears at $k = 4$.
- This makes $k = 4$ a plausible choice, even though the dataset can also be reasonably clustered with $k = 5$.

Elbow method: strengths and limits

- The elbow plot is simple, fast, and easy to explain in teaching and exploratory analysis.
- However, the elbow is often subjective: some curves have no sharp bend or several possible bends.
- For this reason, the elbow method is useful as a first diagnostic, but not as the only decision criterion.

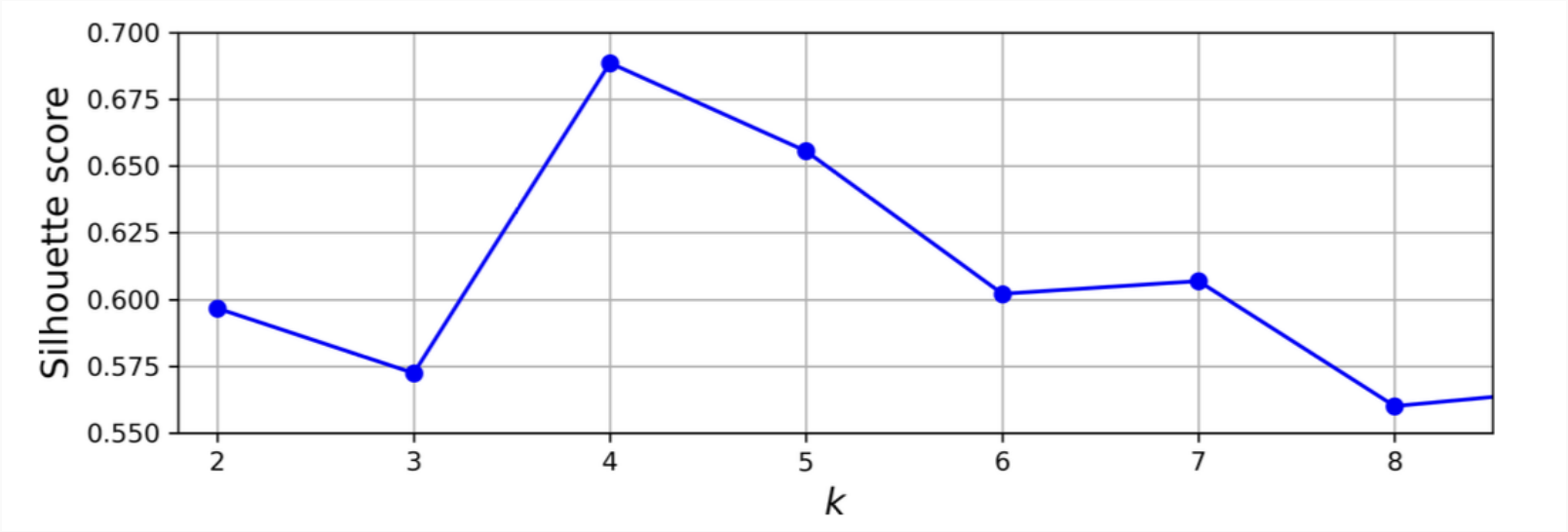
Silhouette coefficient: definition

- For each instance, let a be the mean distance to the other points in the same cluster.
- Let b be the mean distance to the points in the nearest other cluster.
- The silhouette coefficient is

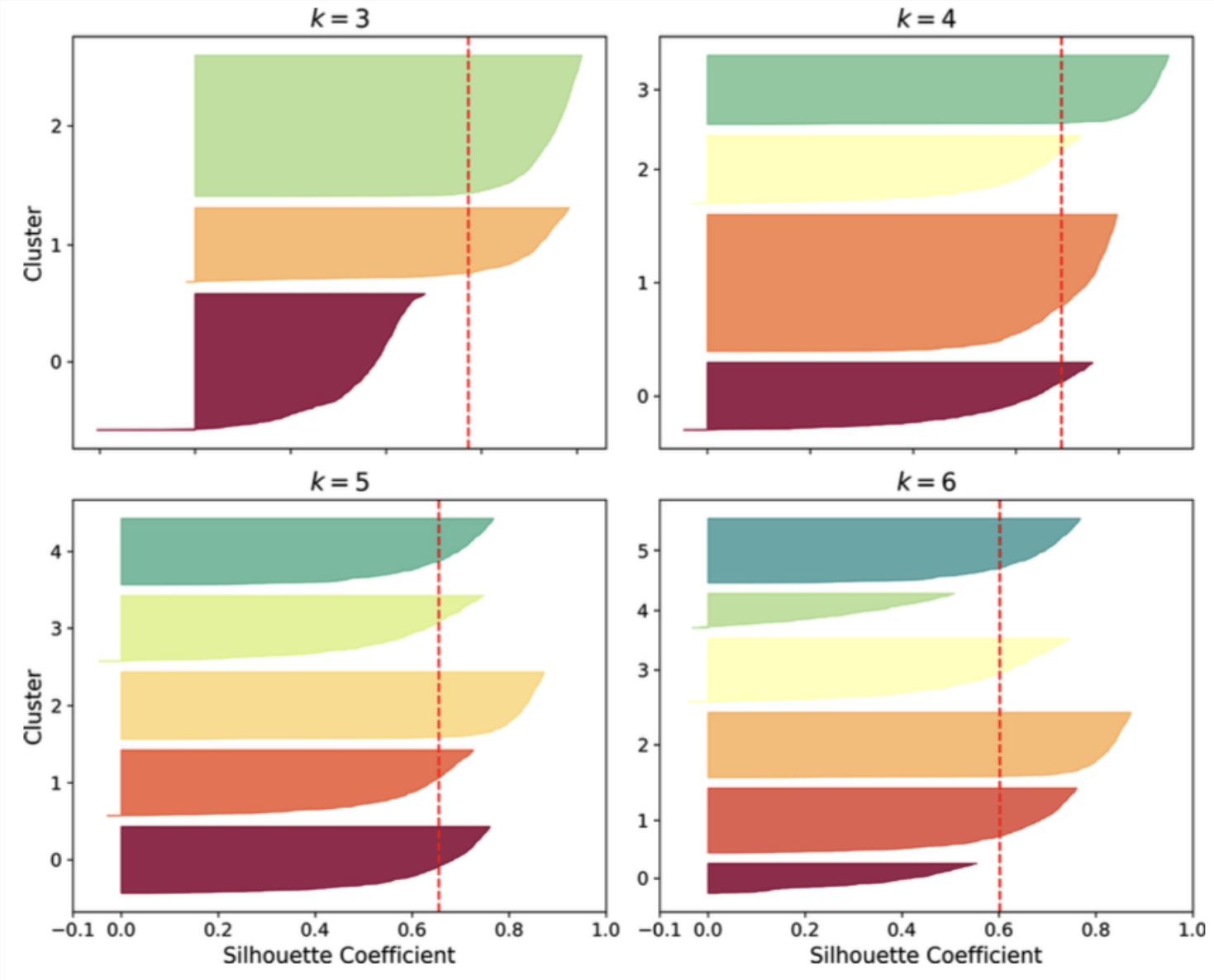
$$s = \frac{b - a}{\max(a, b)}$$

  so it compares within-cluster cohesion with separation from other clusters.

Silhouette coefficient



Silhouette coefficient



Silhouette coefficient: meaning

- A value close to 1 means the point is well inside its own cluster and far from the others.
- A value close to 0 means the point lies near a decision boundary between two clusters.
- A negative value suggests the point may have been assigned to the wrong cluster.

Silhouette score

- The silhouette score is the mean silhouette coefficient over all the instances in the dataset.
- Higher values usually indicate clusters that are both compact and well separated.
- Unlike inertia, the silhouette score can penalize solutions where clusters overlap or where many points sit near boundaries.

Using silhouette to choose k

- Fit k-means for several values of k and compute the silhouette score for each solution.
- Choose values of k that maximize the score, while also checking whether the clusters are balanced and interpretable.
- $k = 4$ has a very good silhouette score, but $k = 5$ is also strong and clearly better than $k = 6$ or $k = 7$.

Why silhouette is richer than the elbow

- The elbow method only tracks compactness through inertia, so it cannot reveal whether clusters are well separated.
- The silhouette score combines compactness and separation, making it more informative for comparing candidate values of k.
- Silhouette diagrams add even more detail, because they reveal cluster imbalance and problematic groups hidden by a single average score.

Practical workflow

- First, scale the features when using k-means, since distance-based clustering is sensitive to feature scales.
- Then inspect the elbow plot to narrow the plausible range of k values.
- Finally, compare silhouette scores and silhouette diagrams to choose a value of k that is both quantitatively strong and substantively useful.
- When the two disagree, inspect the silhouette diagrams and prefer the solution that gives stable, balanced, and interpretable clusters.

Limits of k-means

- k-means works best for roughly spherical clusters with similar size and density.
- It struggles with ellipsoids, uneven densities, and varying cluster diameters because distance to centroids dominates.
- Feature scaling is important; without it, stretched axes can distort the clustering.

k-means failure on ellipsoids

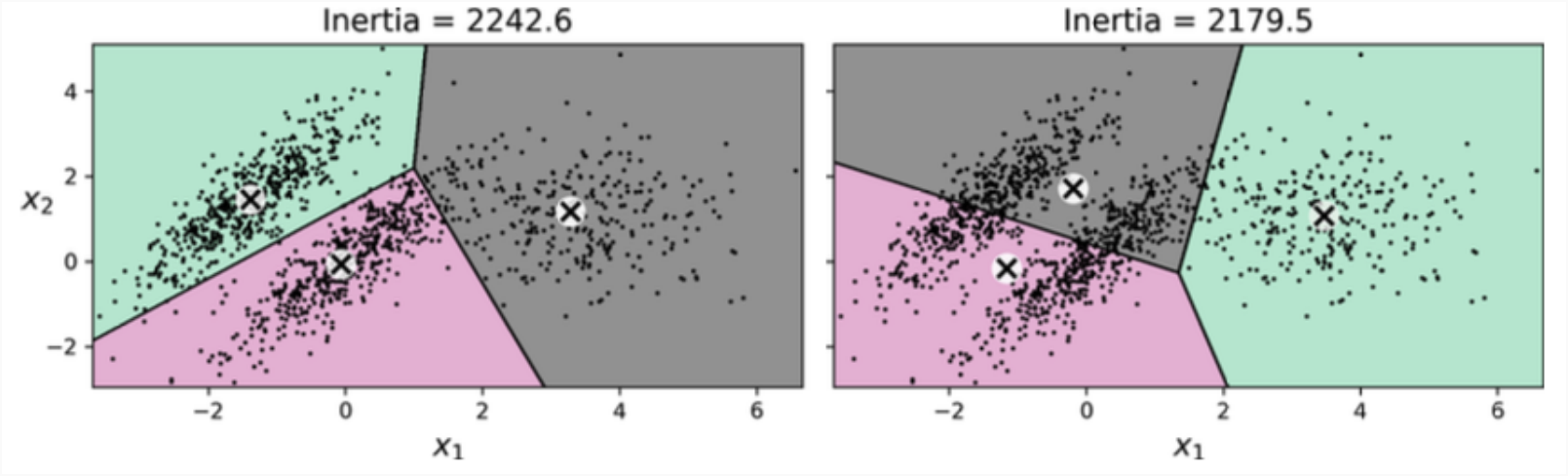
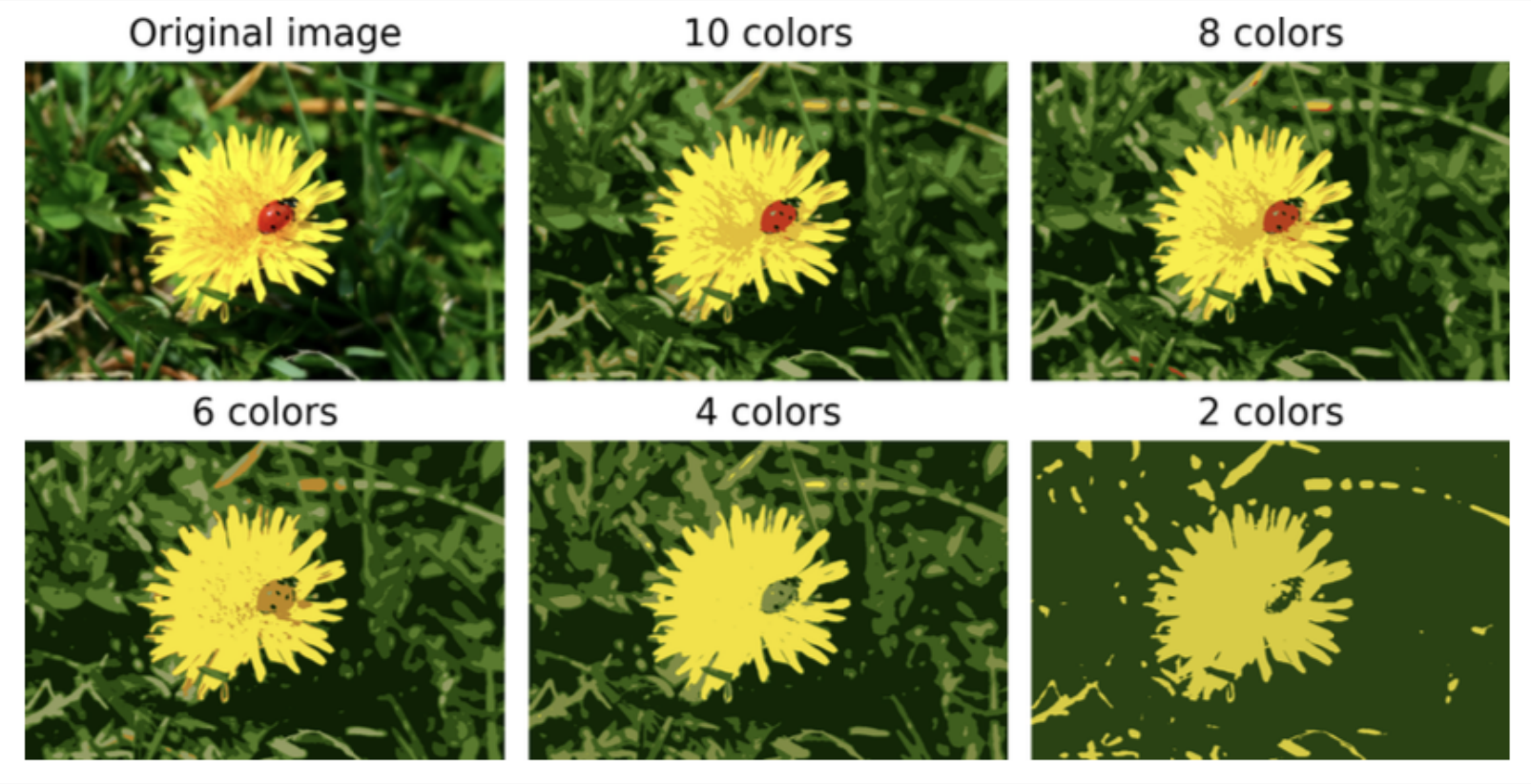


Image segmentation with k-means

- For color segmentation, reshape the image into a list of RGB pixels and cluster colors instead of examples.
- Each pixel is replaced by its cluster centroid color, compressing the palette while preserving structure.
- The chosen number of clusters controls the trade-off between detail and simplification.

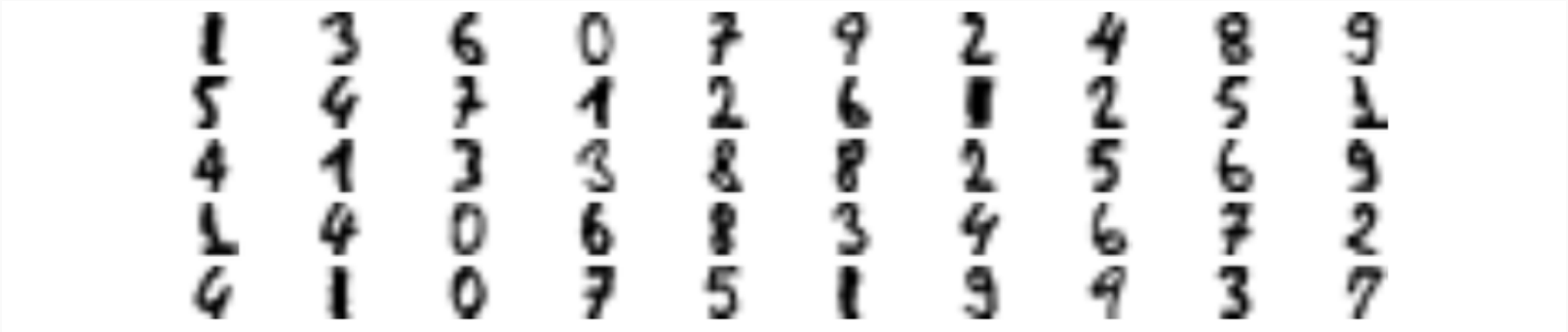
Image segmentation results



Semi-supervised learning

- Cluster the training set, then label only representative instances closest to each centroid.
- This often yields better performance than labeling the same number of random examples.
- Labels can then be propagated inside each cluster to create a much larger pseudo-labeled dataset.

Representative digits



Label propagation workflow

- After clustering, assign each cluster the label of its representative example.
- Optionally discard the farthest instances in each cluster to reduce propagation errors and outlier impact.
- Careful propagation can approach or exceed a fully supervised baseline with few labels.

Active learning

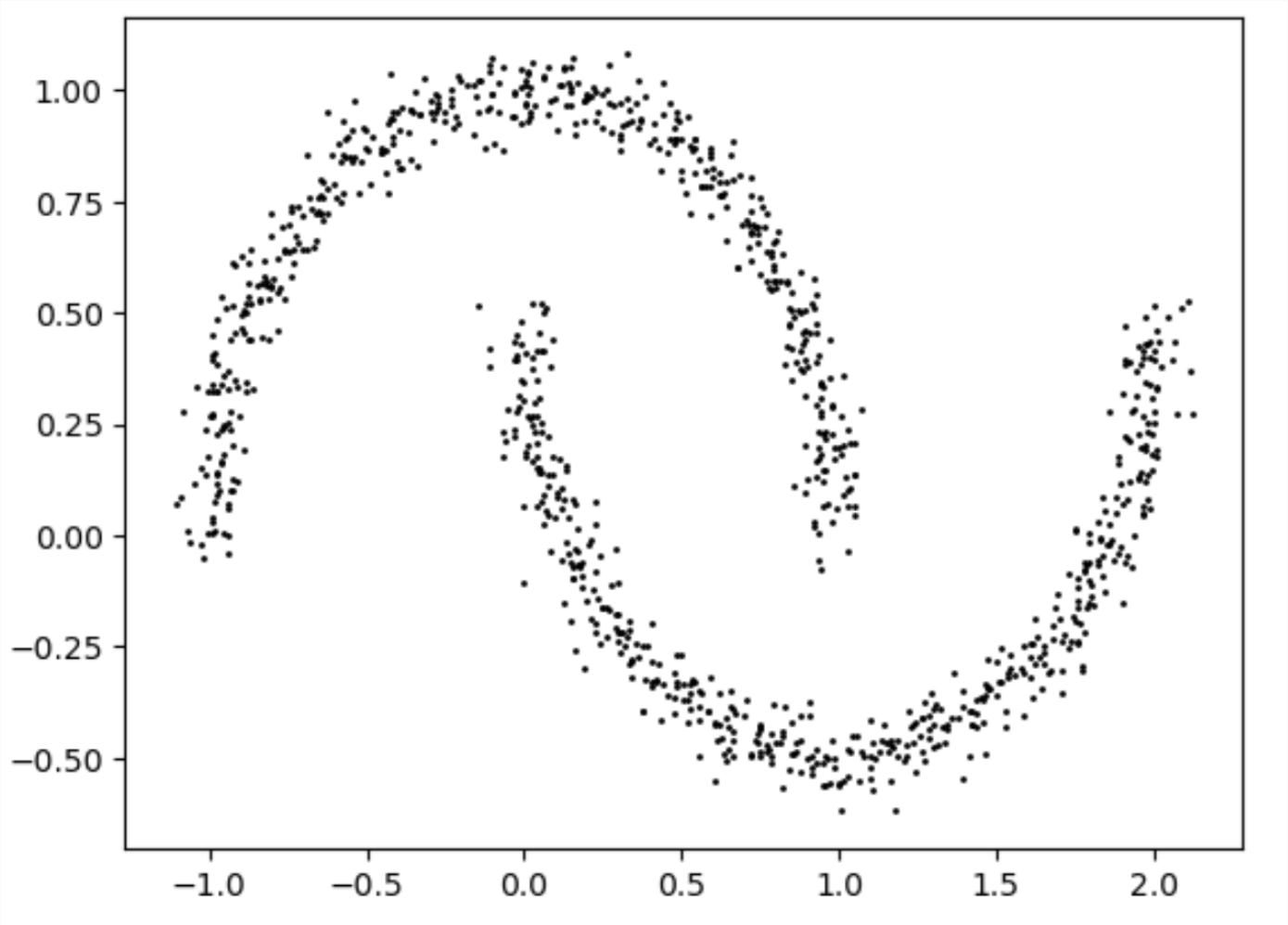
- Active learning lets a human label only the most informative instances requested by the model.
- Uncertainty sampling selects cases for which predicted probabilities are the least confident.
- Other strategies maximize model change, validation-error reduction, or disagreement among models.

DBSCAN

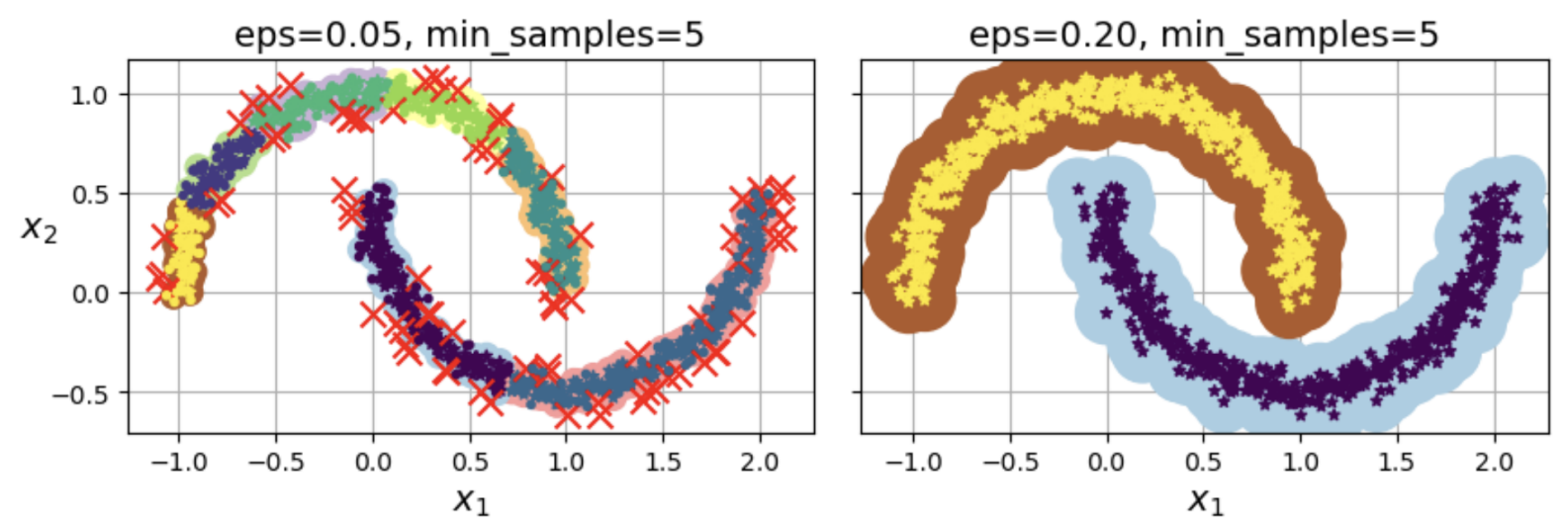
DBSCAN big picture

- **Density-based spatial clustering of applications with noise** (DBSCAN)
- DBSCAN defines clusters as connected high-density regions rather than centroid-centered blobs.
- A point is a **core instance** if at least `min_samples` points lie in its (small) ϵ-neighborhood.
- Points not linked to any dense region are labeled as anomalies.

DBSCAN with two eps values



DBSCAN with two eps values



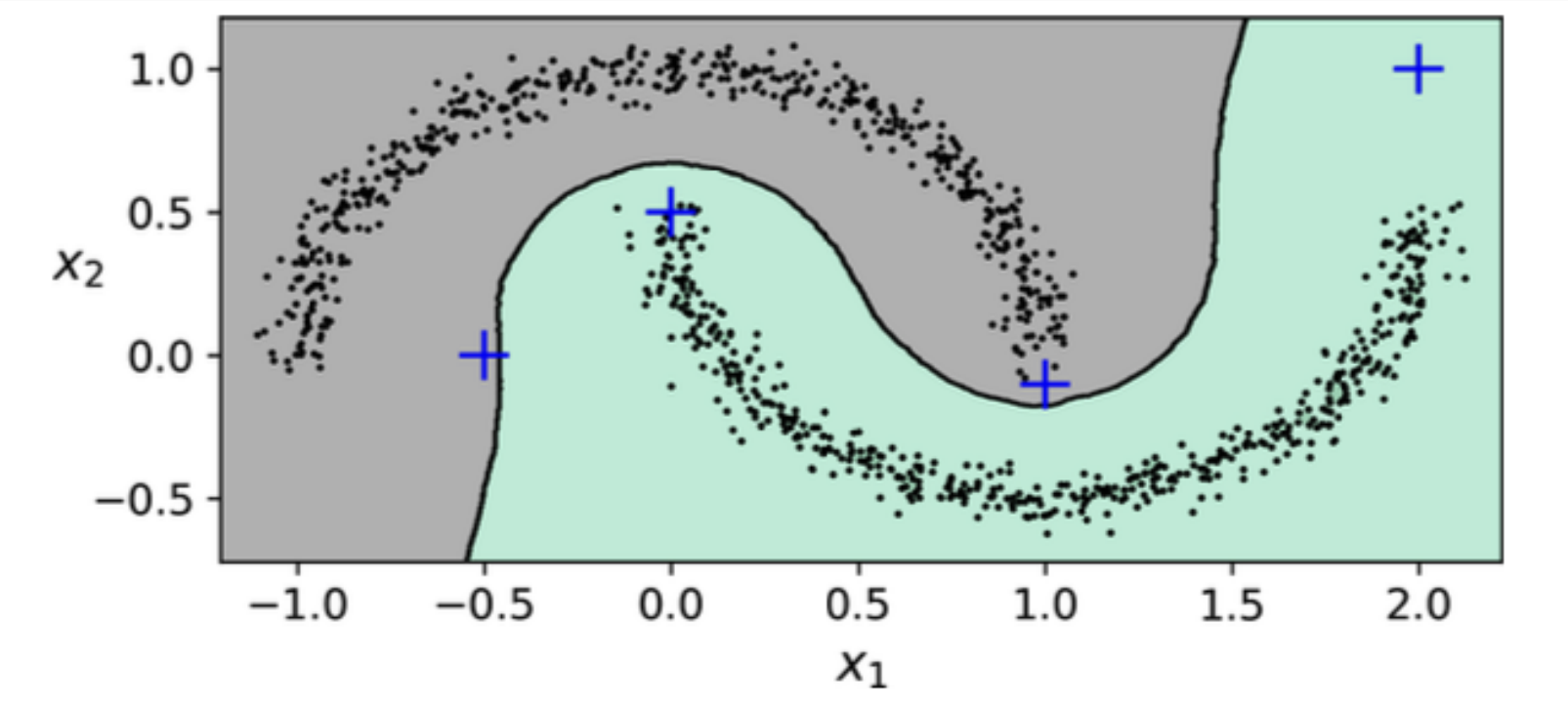
DBSCAN properties

- DBSCAN can discover any number of clusters with arbitrary shapes and is robust to outliers.
- Its main hyperparameters are `eps` and `min_samples`, which control neighborhood size and density threshold.
- It struggles when cluster densities vary strongly or low-density gaps are not clear.

Predicting with DBSCAN

- The Scikit-Learn estimator has `fit_predict` but no native predict method for unseen points.
- One practical workaround is to train a classifier such as k-nearest neighbors on the core samples.
- A distance threshold can then reject faraway samples as anomalies rather than forcing a cluster assignment.

Decision boundary after DBSCAN



Other clustering algorithms

- Agglomerative clustering builds a hierarchy bottom-up and can represent clusters at multiple scales.
- BIRCH is designed for very large datasets; mean-shift and affinity propagation infer cluster count automatically.
- Spectral clustering works from a similarity matrix and is useful for graph cuts or complex manifolds.

Isolation forest

Why isolation forest?

- Isolation Forest is an unsupervised anomaly detection algorithm based on the idea that anomalies are *few and different*, so they are easier to isolate than regular observations.
- Instead of estimating density or distances directly, it repeatedly partitions the feature space at random and measures how fast each sample gets separated.
- It is widely used because it is simple, scalable, and available directly in scikit-learn through the `IsolationForest` estimator.

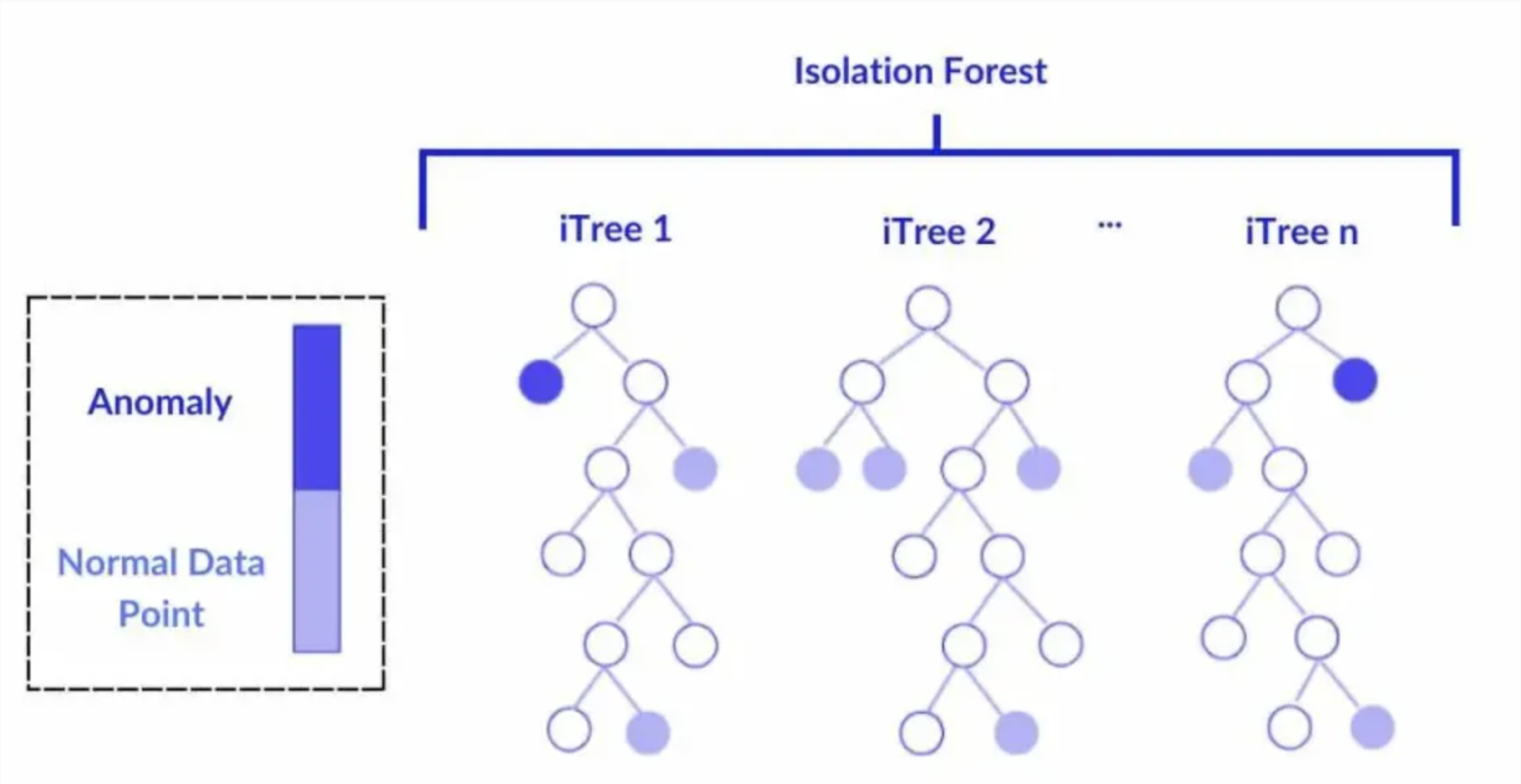
The basic idea

- A normal point lies inside a dense region, so many random splits are usually required before it becomes isolated.
- An anomalous point is often far from the bulk of the data, so a few random splits are enough to isolate it.
- The algorithm therefore uses **path length** in random trees as a proxy for abnormality: shorter path length means more anomalous.

Isolation tree

- An isolation tree starts from all samples at the root node. At each internal node, one feature is selected uniformly at random.
- A split value is then chosen uniformly between the minimum and maximum value of that feature in the current node.
- Recursion stops when a node contains one sample, or all samples in the node are identical, or a maximum depth is reached.

Figure: toy 2D dataset



From one tree to a forest

- A single isolation tree is unstable because it depends on random choices of features and thresholds.
- Isolation Forest averages the path lengths obtained from many isolation trees, reducing variance and producing a more robust anomaly score.
- In scikit-learn, the number of trees is controlled by `n_estimators`, which defaults to 100.

Path length and anomaly score

- For each sample, the model computes its path length in each tree, i.e., the number of splits needed to reach a terminal node.
- The average path length is then normalized using the expected path length of an unsuccessful search in a binary search tree of the same size.
- Samples with short average path length are assigned lower scores and are more likely to be flagged as outliers.

Scikit-learn outputs

- `predict(X)` returns `1` for inliers and `-1` for outliers.
- `decision_function(X)` gives shifted scores: values above 0 are usually treated as inliers, while values below 0 are treated as outliers.
- According to the scikit-learn documentation, `decision_function` $=$ `score_samples` $-$ `offset_`, and when `contamination='auto'` the offset is -0.5.

Key hyperparameters

- `n_estimators`: number of trees in the forest; larger values reduce variance but increase runtime.
- `max_samples`: subsample size used to build each tree; in scikit-learn, `'auto'` means $\min(256, n\_samples)$.
- `contamination`: expected proportion of outliers; this directly affects the threshold used to label observations as anomalies.

More hyperparameters

- `max_features` controls the number of features drawn for each base estimator; reducing it can increase diversity across trees.
- `bootstrap=True` enables sampling with replacement, while the default uses sampling without replacement.
- `random_state` should be fixed when reproducibility matters, especially for teaching material and figures.

Strengths

- Isolation Forest is computationally efficient because it avoids expensive pairwise distance calculations and explicit density estimation.
- It works well in high-dimensional settings and on large datasets, which is one reason it is popular in practice.
- It often provides a strong baseline for unsupervised anomaly detection before trying more specialized methods.

Limitations

- The method assumes anomalies are easier to isolate; this may fail when anomalies form dense clusters of their own.
- Scores are relative to the observed data distribution, so changing the dataset or the contamination assumption can change the threshold substantially.
- Like many anomaly detectors, it may be sensitive to irrelevant features and to scaling differences across dimensions.

Outlier detection vs novelty detection

- In classical outlier detection, the training data may already contain anomalies, and the model tries to identify them during fit and prediction.
- In novelty detection, the model is fit on mostly clean normal data and is later used to reject new abnormal observations.
- Small synthetic 2D examples are useful to visualize how Isolation Forest separates low-score regions from normal regions.

Figure: toy 2D dataset

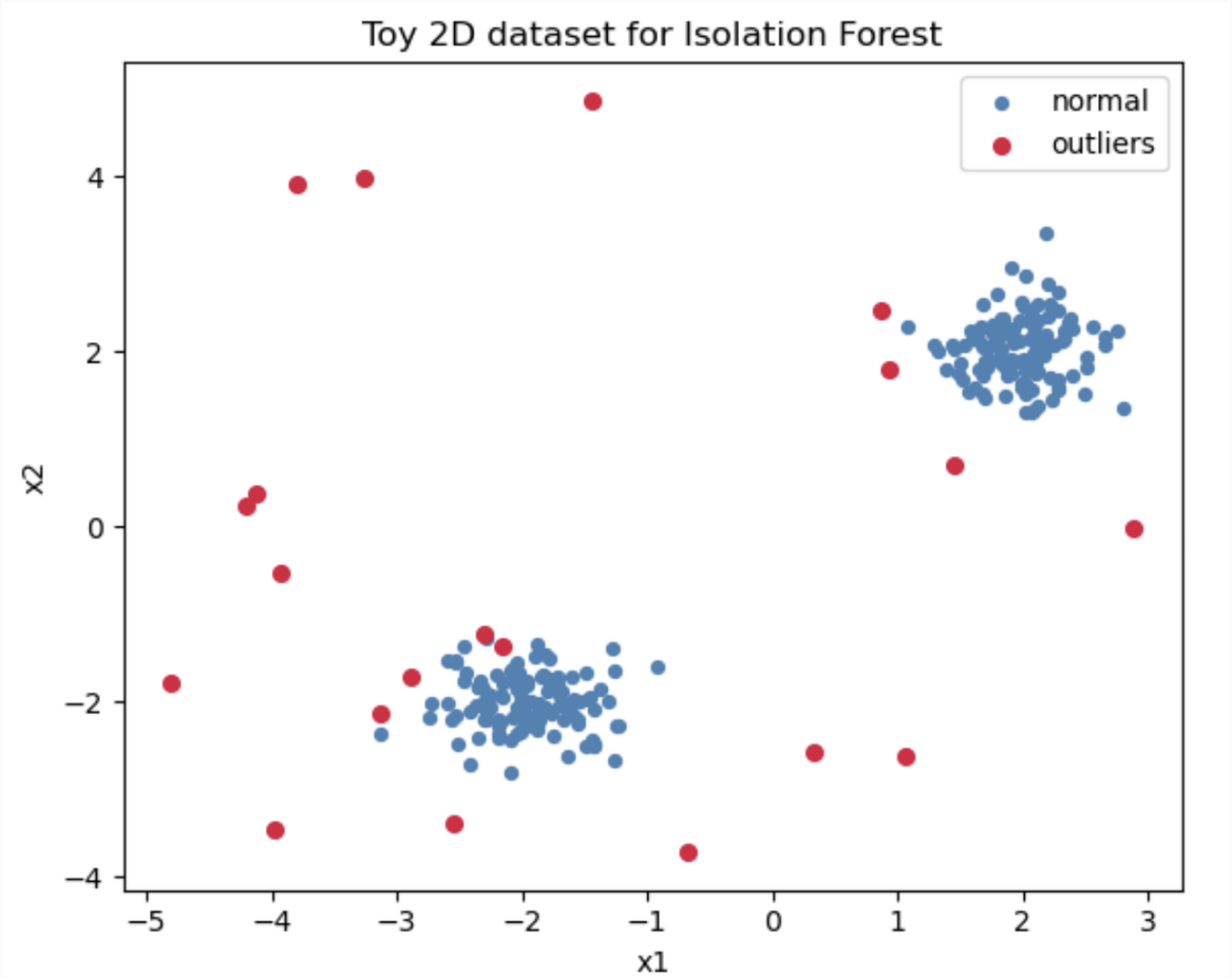
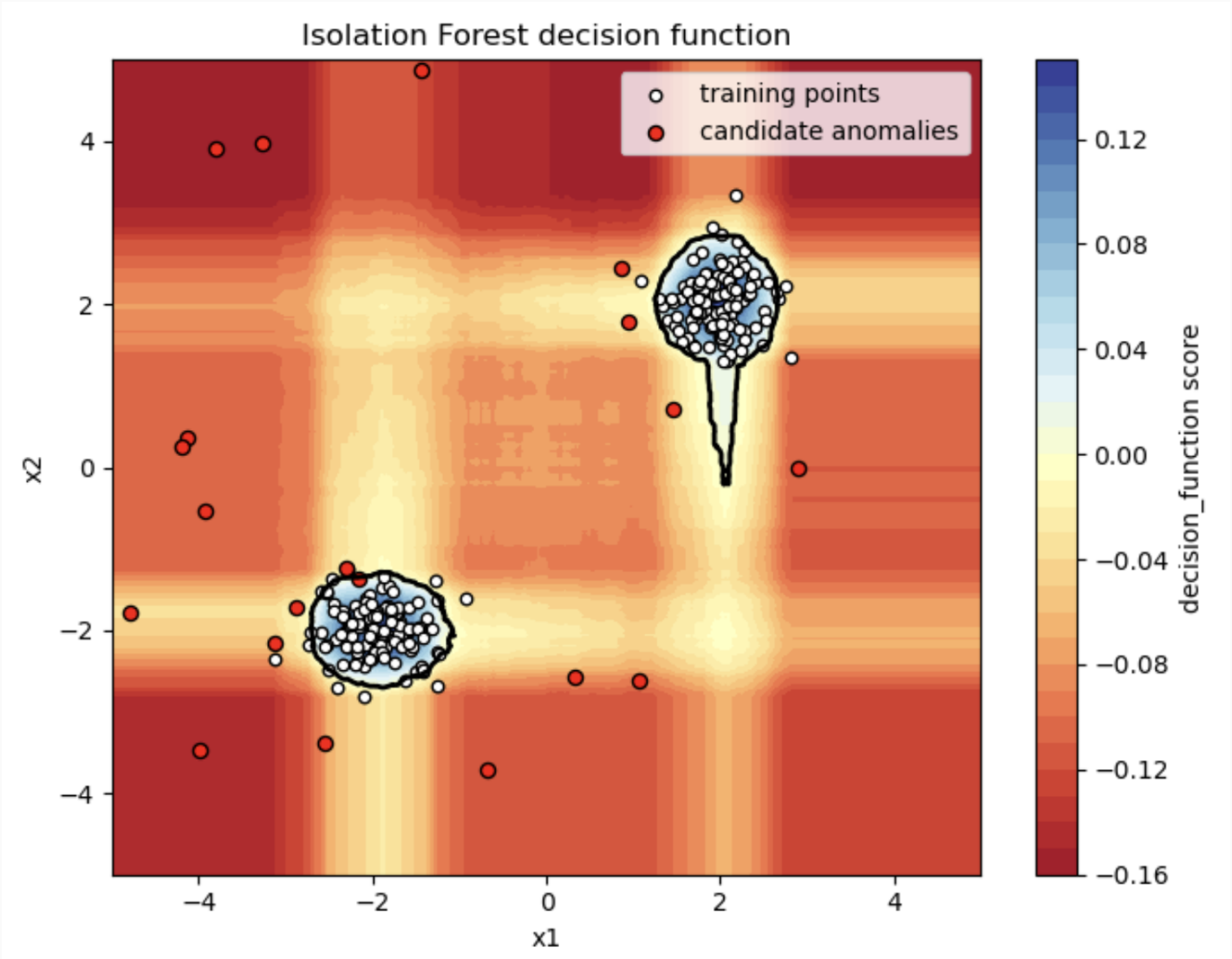


Figure: decision scores in 2D



Reading the decision boundary

- The background color represents the value of the decision function over a grid in the 2D feature space.
- The contour level at 0 acts as the operational boundary between inliers and outliers in the scikit-learn implementation.
- Points lying in regions with strongly negative scores are much more likely to be isolated quickly by the forest.

Comparison with distance-based methods

- Distance-based methods flag points that are far from neighbors, while Isolation Forest focuses on how quickly points can be separated by random splits.
- This makes Isolation Forest attractive when density estimation is difficult or expensive.
- However, local-density methods such as LOF can be better when the notion of anomaly is strongly local rather than global.

Comparison with one-class models

- One-Class SVM learns a boundary around normal data, whereas Isolation Forest estimates abnormality through average isolation depth.
- Isolation Forest is usually easier to scale and tune on large datasets.
- One-Class SVM can model complex nonlinear frontiers, but it is more sensitive to kernel and scale choices.

Typical workflow

- Prepare the feature matrix, handle missing values, and scale features if different magnitudes would distort the splits.
- Fit `IsolationForest` with a reasonable `contamination` guess or with `'auto'` for exploratory analysis.
- Inspect anomaly scores, predicted labels, and flagged samples instead of relying only on the binary output.

Practical advice

- Use more than one diagnostic: histograms of scores, 2D projections, and domain-level inspection of flagged observations.
- Treat `contamination` as a business or domain decision, not just a technical knob.
- When labels are available on a validation subset, use them to calibrate the threshold and compare against alternative detectors.

Summary points

- Isolation Forest detects anomalies by measuring how quickly observations are isolated in random trees.
- Its central signal is the average path length across many isolation trees.
- It is easy to use, fast to train, and especially useful as a first anomaly detection baseline.

Gaussian mixtures

Gaussian mixtures

- A Gaussian Mixture Model assumes data is generated by a weighted sum of Gaussian distributions.
- Each component defines a cluster with its own mean, covariance, and relative weight.
- Unlike k-means, clusters can be ellipsoidal and assignments are probabilistic.

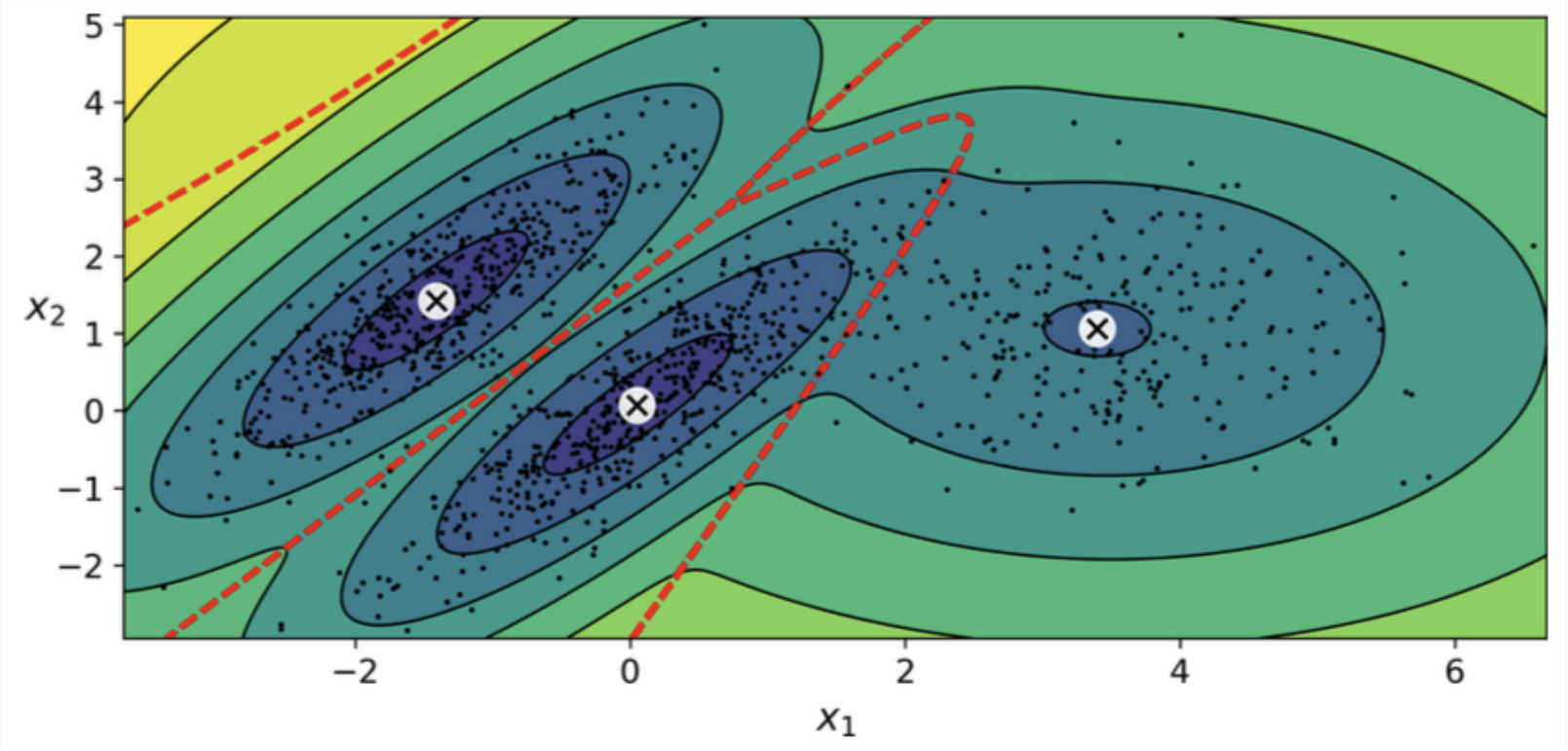
Expectation-maximization

- EM alternates between estimating responsibilities for each component and updating model parameters.
- This generalizes k-means from hard assignments to soft assignments with learned shapes and orientations.
- As with k-means, multiple initializations may be needed because EM can converge to local optima.

GMM outputs

- A trained model exposes component weights, means, and covariance matrices.
- Hard assignments come from predict, while predict_proba returns soft cluster memberships.
- Because GMMs are generative, they can also sample synthetic instances from the learned mixture.

GMM contours and boundaries



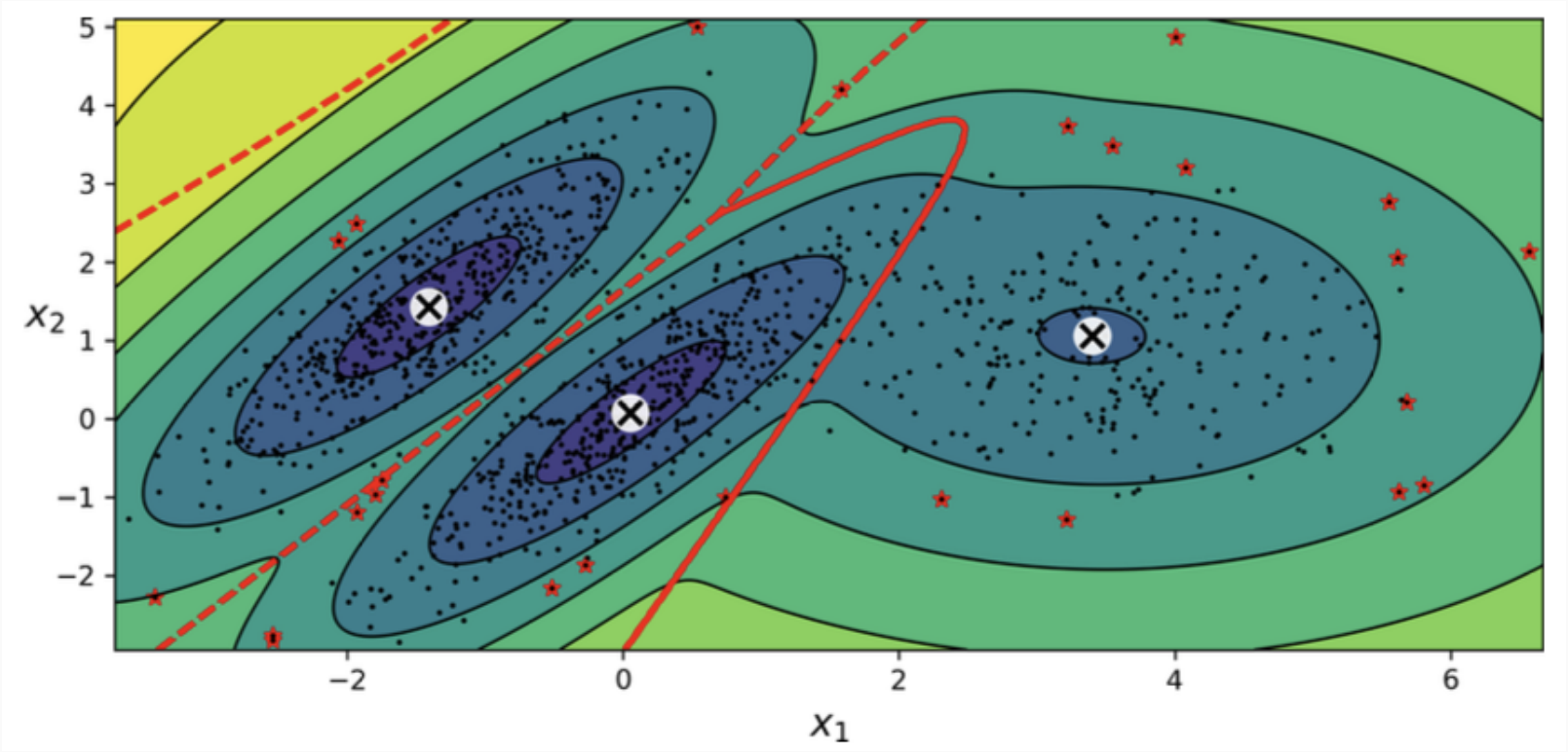
Choosing covariance type

- Spherical enforces round clusters, diag allows axis-aligned ellipsoids, and tied shares one covariance across clusters.
- Full covariance is most flexible but also the most parameter-heavy and computationally expensive.
- Restricting covariance can stabilize learning when data are scarce or dimensionality is high.

GMM for anomaly detection

- Low-density regions under the learned mixture correspond to unusual or potentially defective instances.
- Choose a density threshold from domain knowledge, such as flagging the lowest 2 percent of scores.
- This creates the standard precision-recall trade-off between false alarms and missed anomalies.

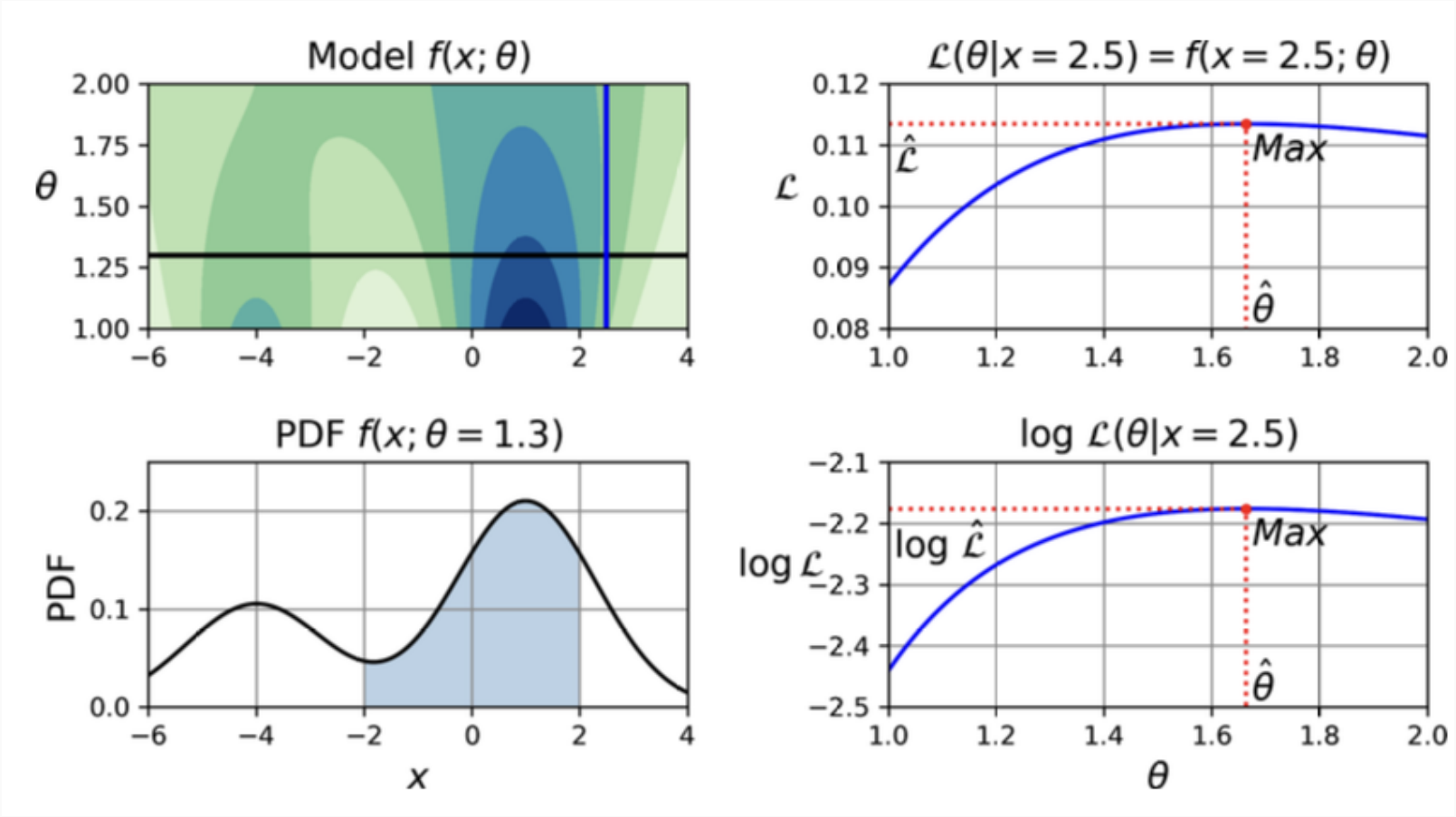
Anomaly detection with GMM



Likelihood, AIC, and BIC

- Likelihood measures how plausible observed data are under particular parameter values.
- AIC and BIC penalize model complexity while rewarding better fit, making them useful to select the number of components.
- BIC typically favors simpler models, especially as dataset size grows.

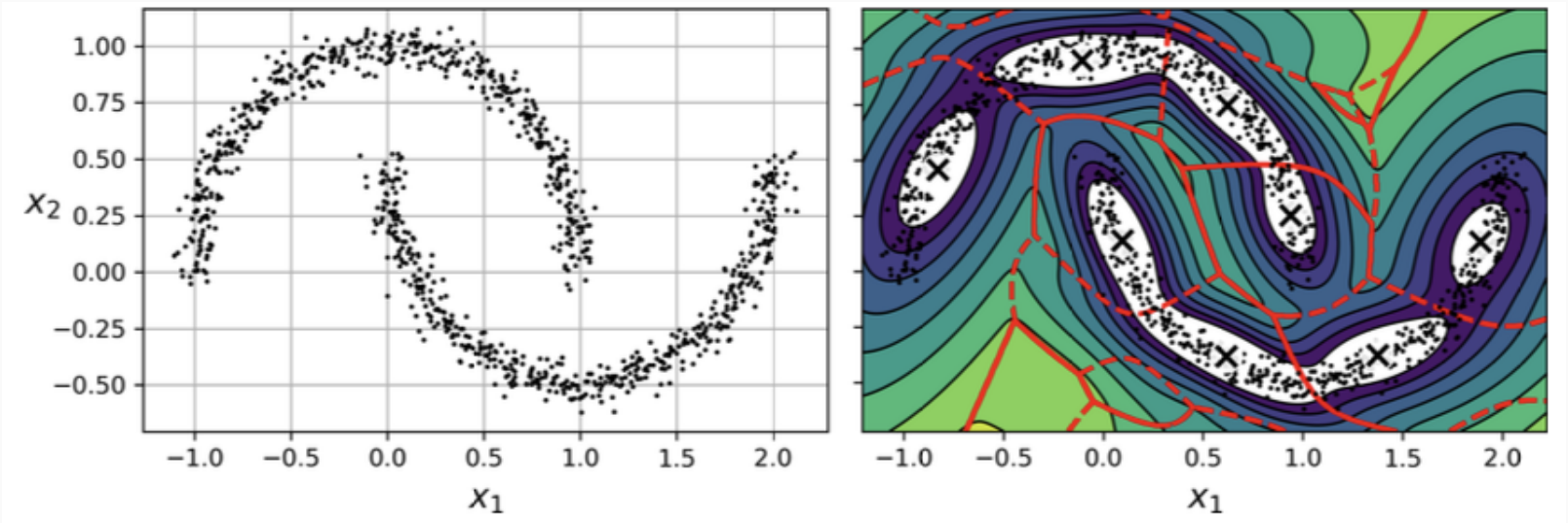
Likelihood function



Bayesian Gaussian mixtures

- BayesianGaussianMixture starts with more components than needed and can shrink unnecessary weights toward zero.
- This reduces the need for manual search over the exact number of clusters.
- It still works best when cluster shapes are reasonably ellipsoidal.

Bayesian GMM on nonellipsoidal data



Anomaly and novelty tools

- EllipticEnvelope fits a robust Gaussian envelope for outlier detection in mostly Gaussian data.
- Isolation Forest isolates anomalies quickly and scales well in high-dimensional settings.
- Local Outlier Factor, one-class SVM, and reconstruction error from PCA are additional alternatives.

Conclusions

- Use k-means for fast, scalable centroid-based clustering on roughly spherical groups.
- Use DBSCAN for arbitrary shapes and explicit noise handling when densities are reasonably consistent.
- Use Gaussian mixtures when soft assignments, density estimates, or ellipsoidal clusters are important.